

Deliberate Red-Run Evidence & Defect Demonstration Report

Executive Summary

As part of the quality assurance and safety enforcement verification for the **Hostel Outing Permission & Approval Management System**, a deliberate defect was introduced to verify that the automated test suite effectively catches security and workflow rule regressions.

Specifically, the mandatory backend business rule enforcing **Warden Parent Approval Confirmation** (`parent_approval_confirmed == True`) before final Warden approval was temporarily disabled.

1. Intentionally Broken Component & Logic

- **Target File:** `outing_service.py` (file:///c:/Data/Inter%20Assign/backend/app/services/outing_service.py)
- **Target Function:** `OutingService.warden_approve`
- **Defect Description:** The backend check verifying `outing.parent_approval_confirmed` was commented out, allowing a Warden to grant final approval directly without confirming parent consent first.

Code Modification Diff (Defect Injection)

```
def warden_approve(db: Session, outing_id: int, warden_user: User, comment: Optional[str] = None):
    outing = db.query(OutingRequest).filter(OutingRequest.id == outing_id).first()
    if outing.status != OutingStatus.PENDING_WARDEN:
        raise HTTPException(status_code=400, detail="...")

-     if not outing.parent_approval_confirmed:
-         raise HTTPException(
-             status_code=400,
-             detail="Parent approval has not been confirmed. Warden cannot give final approval without parent confirmation."
-         )
+     # TEMPORARILY DISABLED FOR DELIBERATE RED RUN DEMONSTRATION
+     # if not outing.parent_approval_confirmed:
+     #     raise HTTPException(...)
```

2. Expected Failure Rationale

The business requirement mandates that:

1. No parent portal exists.
2. The Warden personally verifies parent approval by phone/in person.
3. The Warden UI requires checking [] Parent approval obtained.
4. The backend MUST reject final approval (HTTP 400 Bad Request) if parent approval has not been confirmed.

When this check is removed, an unconfirmed request receives an HTTP 200 OK status change to APPROVED instead of being blocked with HTTP 400 Bad Request.

3. Test Execution Output (RED RUN FAILURE)

Execution Command

```
.\backend\venv\Scripts\pytest -k test_16_warden_cannot_approve_without_parent_confirmation
```

Captured Terminal Output (Failure)

```
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Data\Inter Assign
configfile: pytest.ini
testpaths: backend/tests
plugins: anyio-4.14.2, asyncio-1.4.0
collected 30 items / 29 deselected / 1 selected

backend\tests\test_workflow.py F [100%]

===== FAILURES =====
_____ test_16_warden_cannot_approve_without_parent_confirmation _____

def test_16_warden_cannot_approve_without_parent_confirmation(client, seed_users):
    student1 = seed_users["student1"]
    hod = seed_users["hod"]
    warden = seed_users["warden"]
    h_student = get_auth_header(student1)
    h_hod = get_auth_header(hod)
    h_warden = get_auth_header(warden)
    tomorrow = (date.today() + timedelta(days=1)).isoformat()

    create_resp = client.post("/api/outings", json={}, headers=h_student)
    outing_id = create_resp.json()["id"]

    client.post(f"/api/hod/outings/{outing_id}/approve", json={"comment": "HOD approved"}, headers=h_hod)

    # Warden tries to approve WITHOUT parent confirmation
    approve_resp = client.post(f"/api/warden/outings/{outing_id}/approve", json={"comment": "Direct approve attempt"}, headers=h
>     assert approve_resp.status_code == 400
E       assert 200 == 400
E         + where 200 = <Response [200 OK]>.status_code

backend\tests\test_workflow.py:333: AssertionError
===== short test summary info =====
FAILED backend/tests/test_workflow.py::test_16_warden_cannot_approve_without_parent_confirmation
===== 1 failed, 29 deselected in 0.50s =====
```

4. Resolution & Restoration

The mandatory validation was restored in `outing_service.py` (file:///c:/Data/Inter%20Assign/backend/app/services/outing_service.py):

```
# MANDATORY CHECK: Parent approval MUST be explicitly confirmed before Warden final approval
if not outing.parent_approval_confirmed:
    raise
    HTTPException( status_code=status.HTTP_400_BAD_REQUEST,
        detail="Parent approval has not been confirmed. Warden cannot give final approval without parent confirmation."
    )
```

5. Final Verification Test Result (GREEN PASS)

Execution Command

```
.\backend\venv\Scripts\pytest -v
```

Captured Terminal Output (All Passed)

```
===== test session starts =====
platform win32 -- Python 3.12.0, pytest-9.1.1, pluggy-1.6.0 -- C:\Data\Inter Assign\backend\venv\Scripts\python.exe
testpaths: backend/tests
collected 30 items

backend/tests/test_workflow.py::test_01_student_login_works PASSED      [ 3%]
backend/tests/test_workflow.py::test_02_invalid_login_fails PASSED      [ 6%]
backend/tests/test_workflow.py::test_03_student_can_create_outing PASSED [ 10%]
...
backend/tests/test_workflow.py::test_16_warden_cannot_approve_without_parent_confirmation PASSED [ 53%]
...
backend/tests/test_workflow.py::test_30_history_cannot_be_modified_by_normal_user_endpoints PASSED [100%]

===== 30 passed, 1 warning in 7.44s =====
```

Conclusion: The deliberate red-run successfully proved that `test_16_warden_cannot_approve_without_parent_confirmation` strictly protects business rules against authorization bypasses.